

Cooling Section

11

For numerical work we use NumPy arrays

```
import numpy as np
x = np.array([1., 2., 3., 4.])
```

Important property:

$x.shape$ # dimensions, $x.shape[0]$ and $x.shape[1]$

These two will occur constantly:

- $x = np.arange(0, 10, 1)$
- $x = np.linspace(0, 1, 101)$ # to get a grid ← fancy way of saying we get some values

Broadcasting

We have $x = np.array([1., 2., 3.])$, then $x.shape$ is $(3,)$

↑ 1D array with 3 elements

→ We do $x[:, None].shape$ and $x[None, :].shape$ to get $(3, 1)$ and $(1, 3)$ respectively.

↳ so $x[:, None]$ is a row vector and $x[None, :]$ is a column vector

- $None$ and $np.newaxis$ do the same

Example

$x = np.array([1., 2., 3.])$ and $y = np.array([10., 20.])$ and we compute $x+y$.

We cannot directly add these so what we can do is:

$x[:, None] + y[None, :]$

Tracing the shapes: $(3, 1) + (1, 2) \rightsquigarrow (3, 2) \Rightarrow \begin{bmatrix} 11 & 21 \\ 12 & 22 \\ 13 & 23 \end{bmatrix}$

▼ NumPy compares dimensions from right to left. Two dimensions are compatible if they are equal or one of them is equal to 1.

↓

$(4, 3)$ and $(4, 3)$
 $(3,)$ and $(3, 1)$
this would then be $(1, 3)$ as missing dims are added on the left. → usage of $np.newaxis$

Vectorisation

We want $y_i = x_i^2 + 2x_i + 1$ for 10^6 distinct values.

• Python loop version

```
y = np.empty(len(x))
for i in range(len(x)):
    y[i] = x[i]**2 + 2*x[i] + 1
```

• Vectorized version

```
y = x**2 + 2*x + 1
```

but vectorized version is faster

→ Vectorization doesn't really improve asymptotic complexity but it improves the constant factor

→ Reminder from W1 Notes: Computational compl. $\neq$ Execution time

→ it touches to things like SIMD, optimized mem. access etc.

Code Example 1

Example: Combining shapes + vectorization + broadcasting

$x = [x_0, x_1, \dots, x_m]$, and we want Vandermonde Matrix

```
⇒ powers = np.arange(n+1)
V = x[:, None] ** powers[None, :]
```

⇒ Look at the shapes: $x[:, None] \rightarrow (m+1, 1)$
 $powers[None, :] \rightarrow (1, n+1)$

⇒ Broadcast: $\begin{bmatrix} x_0 & x_0 & \dots \\ x_1 & x_1 & \dots \\ \vdots & \vdots & \ddots \\ x_m & x_m & \dots \end{bmatrix}$ and $\begin{bmatrix} 0 & 1 & 2 & \dots & n \\ 0 & 1 & 2 & \dots & n \\ \vdots & \vdots & \vdots & \ddots & \vdots \end{bmatrix}$

- $*$ is elementwise mult.; $@$ is matrix mult. (for vectors dot product)
- $A.reshape(m, n)$; $A.flatten()$ or $A.ravel()$
search cases for reshape look at the difference between a copy and a view
- $0.1 + 0.2 == 0.3$ NO! $\Rightarrow$ use `np.isclose(0.1 + 0.2, 0.3)`

Coding: Solving Systems

We want to solve $Ax = b$; do `np.linalg.solve(A, b)`.

→ Do not do $x = A^{-1}b$ (`np.linalg.inv(A) @ b`) because constructing the inverse is generally not numerically desirable.

Chapter 2 : Polynomial Interpolation

For some function $f(x)$ we could again have that its analytical expression is unknown / complicated or evaluating it is expensive.

In the usual setting we have data points.

$$\begin{bmatrix} x_0 & x_1 & \dots & x_n \\ y_0 & y_1 & \dots & y_n \end{bmatrix} \text{ with } x_i, y_i \in \mathbb{R}$$

$\Rightarrow x_i$'s are nodes / support points and $y_i = f(x_i)$'s are function values

Our goal is to construct a simpler $\tilde{f}$ s.t. $\tilde{f}(x_i) = y_i$

True function f could belong to some large or infinite-dim. function space V . We can choose a $V_n \subset V$ with a basis $b_1, \dots, b_n$. Then we get:

$$f(x) \approx f_n(x) = \sum_{j=1}^n \alpha_j b_j(x)$$

Observe now the entire unknown function is represented by finitely many numbers (e.g. $\alpha = [\alpha_1, \dots, \alpha_n]$)

Code Example 2

Theorem 2.1.5 (Peano)

If f is continuous, then there exists a polynomial p that approx. f arbitrarily well w.r.t the infinity norm:

$\|f - p\|_\infty$ can be made arb. small, where $\|f - p\|_\infty = \max_x |f(x) - p(x)|$

Remark 2.1.6

In general, the smoother f is, the better convergence we can expect from the approximation methods considered in the course

Polynomials and monomial Basis

Space of poly. of degree at most k is $P_k = \{p(x) = \alpha_k x^k + \alpha_{k-1} x^{k-1} + \dots + \alpha_1 x + \alpha_0\}$ and α 's could be from $\mathbb{R}$ or $\mathbb{C}$

$\Rightarrow$ here we see that P_k is made out of monomials of degree at most k
 $x \mapsto x^k \quad k \in \mathbb{N} \cup \{0\}$

$\Rightarrow$ Observe $\{1, x, \dots, x^k\}$ forms a basis of P_k , there are $k+1$ indep. functions

→ This also means $\dim P_k = k+1$

Theorem 2.1.10 Uniqueness

Given $k+1$ distinct nodes $x_0, \dots, x_k$ a poly of degree at most k is uniquely determined by $p(x_i) = y_i$, $i = 0, \dots, k$.

Extra Proof

Let $p, q \in P_k$ be arbitrary but distinct s.t. $p(x_i) = q(x_i) = y_i$.

Define $r(x) = p(x) - q(x)$ then $r \in P_k$ as well, that gives us at every interpolation node:

$$r(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0$$

Therefore r has $k+1$ distinct roots $x_0, x_1, \dots, x_k$, but a poly. of degree at most k can have at most k distinct roots.

Hence $r(x) \equiv 0 \Rightarrow p = q$

Example 2.1.11

$$\begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \begin{bmatrix} \alpha_0 \\ \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_n \end{bmatrix} \Leftrightarrow V\alpha = y$$

for some x and y :

```
V = np.vander(x, N=N+1, increasing=True)
alpha = np.linalg.solve(V, y)
```

↳ so here interpolation becomes a linear system for the coefficients

Code expert: Newton Interpolation polynomial

⇒ First we run on inspect the plot.

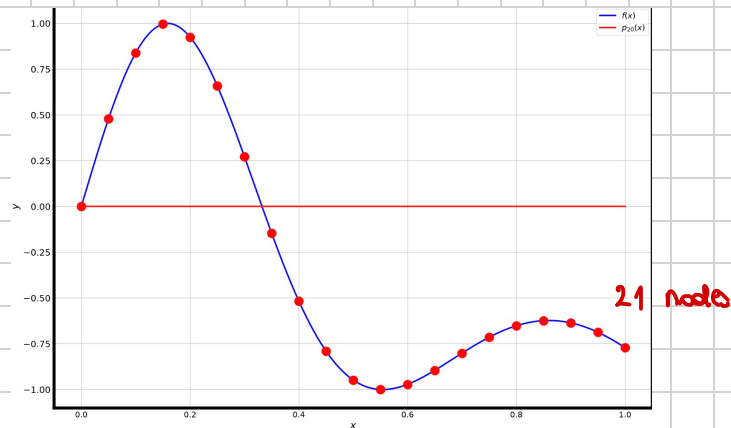
```

n = 20
x = np.linspace(0,1,endpoint=True, num=n+1)
f = lambda x: np.sin(10*x*np.cos(x))
yx = f(x)
y = yx.copy()
dd = divdiff(x, y)
N = 1000
t = np.linspace(0,1,endpoint=True, num=N+1)

yt = evalNewton(x, dd, t)

```

equal
eval. points



⇒ f is the true function

What is Newton Interpolation polynomial?

Special thing is that it allows us to add new nodes without changing the previous nodes.

$n=2$:

$n=3$:

$$p_1(x) = \beta_0 + \beta_1(x-x_0) \rightarrow \text{we want } p_2(x) = p_1(x) + \beta_2(x-x_0)(x-x_1)$$

now observe at x_0 and x_1 the new term $\beta_2(x-x_0)(x-x_1)$ is 0.

Newton Basis

$$N_0(x) = 1$$

$$N_1(x) = x - x_0$$

$$N_2(x) = (x - x_0)(x - x_1)$$

and in general

$$N_j(x) = \prod_{i=0}^{j-1} (x - x_i)$$

p in Newton Basis

$$p_n(x) = \beta_0 N_0(x) + \beta_1 N_1(x) + \beta_2 N_2(x) + \dots + \beta_n N_n(x)$$

⇒ The goal is then to calculate β_i 's ... the divided differences

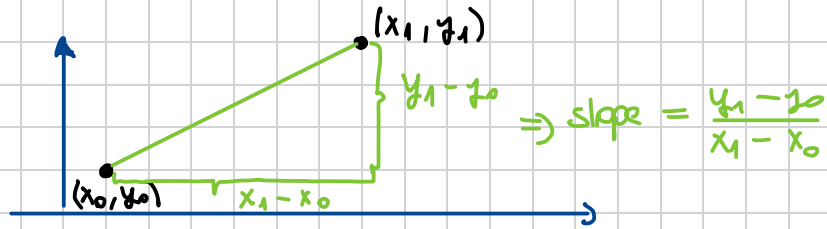
Examples:

$n=1$: $p_1(x) = \beta_0 N_0(x) = \beta_0 \Rightarrow$ As $p(x_0) = y_0$ must hold we get $\beta_0 = y_0$

$n=2$: $p_2(x) = \beta_0 + \beta_1 N_1(x) = y_0 + \beta_1 (x - x_0)$

⇒ Since $p_2(x_1) = y_1 \Rightarrow y_0 + \beta_1 (x_1 - x_0) = y_1$

⇒ $\beta_1 = \frac{y_1 - y_0}{x_1 - x_0} \dots$ div. diff.



$n=3$: $p_3(x) = y_0 + \left(\frac{y_1 - y_0}{x_1 - x_0}\right)(x - x_0) + \beta_2(x - x_0)(x - x_1) = y_2$

⋮

$$\Rightarrow \beta_2 = \frac{\frac{y_1 - y_0}{x_1 - x_0} - \frac{y_2 - y_1}{x_2 - x_1}}{x_2 - x_0} = \frac{y[x_0, x_1] - y[x_1, x_2]}{x_2 - x_0}$$

In general:

$$y[x_i, \dots, x_{i+k}] = \frac{y[x_{i+1}, \dots, x_{i+k}] - y[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}$$

We use this in our code!

$$\begin{array}{l|l} x_0 & y[x_0] \xrightarrow{\beta_0} > y[x_0, x_1] \\ x_1 & y[x_1] \xrightarrow{\beta_1} > y[x_0, x_1, x_2] \\ x_2 & y[x_2] \xrightarrow{\beta_2} > y[x_0, x_1, x_2, x_3] \\ x_3 & y[x_3] \xrightarrow{\beta_3} > y[x_1, x_2, x_3] \end{array} \quad (2.2.7)$$

for each we have to also do some side calculations

Coding tips divdiff - sqw

- having a working code for β_2 should work for all cases, if implemented correctly
- think about how can I reuse things, maybe storing them in an array should help
- implement smt that when you try to compute β_3 , as you already have β_2 from earlier it must only depend on the lower part of the branch in the schema.

Coding tips for divdiff

- Look at the schema:

$$\begin{array}{l} y[x_0, x_1] \Rightarrow \frac{y_1 - y_0}{x_1 - x_0} \\ y[x_1, x_2] \Rightarrow \frac{y_2 - y_1}{x_2 - x_1} \\ y[x_2, x_3] \Rightarrow \frac{y_3 - y_2}{x_3 - x_2} \end{array} \Rightarrow \frac{\begin{pmatrix} y_1 \\ y_2 \end{pmatrix} - \begin{pmatrix} y_0 \\ y_1 \end{pmatrix}}{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - \begin{pmatrix} x_0 \\ x_1 \end{pmatrix}} \Rightarrow \begin{pmatrix} y[x_0, x_1] \\ y[x_1, x_2] \\ y[x_2, x_3] \end{pmatrix}$$

Coding tips for divdiff-rec

- You should make the problem size less at each step until you have only one element:

$\Rightarrow [y_0, y_1, \dots, y_n]$ at the start

$\Rightarrow$ After some steps you have $[y_0]$, observe $y_0 = \beta_0$

$\Rightarrow$ Return it to one above recursive step  $[\beta_0, y_1]$

$\Rightarrow$ Now at step 1 we want to give to step 2 smth like $[\beta_0, \beta_1]$, so that at step 2 it can work with $[\beta_0, \beta_1, y_2]$

Horner Schema and Evaluation

Reminder

$$p(t) = b_0 + b_1(t - x_0) + b_2(t - x_0)(t - x_1) + \dots$$

we can factor things out like $(t - x_0)$ as it reappears at every term after it first occurred.

$$p(t) = b_0 + (t - x_0) [b_1 + (t - x_1) [b_2 + \dots]]$$

This is the newton horner form.

Here we use the Code 2.1.15: Horner schema

Code 2.1.15: Horner-Schema

```
1 def horner(p, x):  
2     y = p[0]  
3     for i in range(1, len(p)):  
4         y = x * y + p[i]  
5     return y
```

$\Rightarrow$ Be careful, here $p = [\alpha_n, \dots, \alpha_0]$
it is backwards



Example

$$xx = [1, 2, 3] \quad dd = [0.5, 1, 2] \quad x = [0, 0.25, 0.10]$$

We need to evaluate $p(t)$ once for each point in xx , as here besides t nothing changes we can easily vectorize it.

e.g.: According to horner schema:

$$\begin{aligned} p(1) &= 0.5 + (1 - 0) [1 + (1 - 0.25) 2] \\ &= b_0 + (t - x_0) [b_1 + (t - x_1) b_2] \end{aligned}$$

Coding tips

- As we are evaluating for more than one point but basically doing the same calculations for each step we can use vectorization
- At the end you should see $p_{20}(x)$ directly on $f(x)$.

Code expert: Polynomial Interpolation eines Polynoms

Theory

Lagrange Polynoms for $x_0, x_1, \dots, x_n \in \mathbb{R}$:

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

⇒ What so special is that?

Take $l_i(x)$ and evaluate it at x_k where $k \neq i$, look at the numerator

↳ $x_k - x_k = 0$, hence $l_i(x_k) = 0$

But for $k=i$ we have:

$$l_i(x_i) = \prod_{j \neq i} \frac{x_i - x_j}{x_i - x_j} = 1 \quad \Rightarrow \quad l_i(x_j) = \begin{cases} 1, & i=j \\ 0, & i \neq j \end{cases}$$

This is like a switch, ith lagrange polynom returns 1 for x_i only! ▼

p in Lagrange Basis

$$p(x) = \sum_{i=0}^n y_i l_i(x)$$

⇒ Here recomputing all products $l_i(x)$ for every evaluation point is very inefficient

Barycentric Form

1. We define $L(x) = \prod_{j=0}^n (x - x_j)$ and **barycentric weights** $\lambda_k = \prod_{j=0, j \neq k}^n \frac{1}{x_k - x_j}$

$$2. l_k(x) = \prod_{j \neq k} \frac{x - x_j}{x_k - x_j} \quad \Rightarrow \quad L(x) = (x - x_k) \prod_{j \neq k} (x - x_j)$$

$$\Leftrightarrow \prod_{j \neq k} (x - x_j) = \frac{L(x)}{(x - x_k)}$$

3. Insert it into $l_k(x) \rightsquigarrow \frac{\prod_{j \neq k} x - x_j}{\prod_{j \neq k} x_k - x_j} = \frac{L(x)}{(x - x_k) \lambda_k^{-1}} = L(x) \frac{\lambda_k}{(x - x_k)}$

4. Insert this into $p(x)$: $p(x) = L(x) \sum_{k=0}^n \frac{\lambda_k y_k}{x - x_k} = \sum_{k=0}^n l_k(x) = 1$

5. Use the 4th property in Remark 2.3.4: and divide

$$\Rightarrow \frac{p(x)}{1} = \frac{L(x) \sum_{k=0}^n \frac{\lambda_k y_k}{x - x_k}}{L(x) \sum_{k=0}^n \frac{\lambda_k}{x - x_k}}$$

$$\Rightarrow \text{At the end you have } p(x) = \frac{\sum_{k=0}^n \frac{\lambda_k y_k}{x - x_k}}{\sum_{k=0}^n \frac{\lambda_k}{x - x_k}}$$

Coding Tips barycentric - weights

- $\lambda_k = \prod_{j \neq k} \frac{1}{x_k - x_j} = \frac{1}{(x_k - x_0) \cdots (x_k - x_{k-1})(x_k - x_{k+1}) \cdots (x_k - x_n)}$
- Use `np.prod(...)`

Coding Tips interp - barycentric

- in $p(x)$ there is the term $x - x_k$, x is here the evaluation points (`xx` in `codexpert`)

→ you can do $z_k = xx_i - x_k$ for one evaluation point xx_i

→ then define $\mu_k = \frac{\lambda_k}{z_k} \Rightarrow p(xx_i) = \frac{\sum_k \mu_k y_k}{\sum_k \mu_k}$

- Be careful in the evaluation points there could be a support point!

Once the nodes are given the interpolation is unique, but not all nodes deliver an equally good approximation.

⇒ Lebesgue constant: for interp. nodes $x_0, \dots, x_n \in [a, b]$

$$\Lambda_n = \max_{x \in [a, b]} \sum_{i=0}^n |l_i(x)|$$

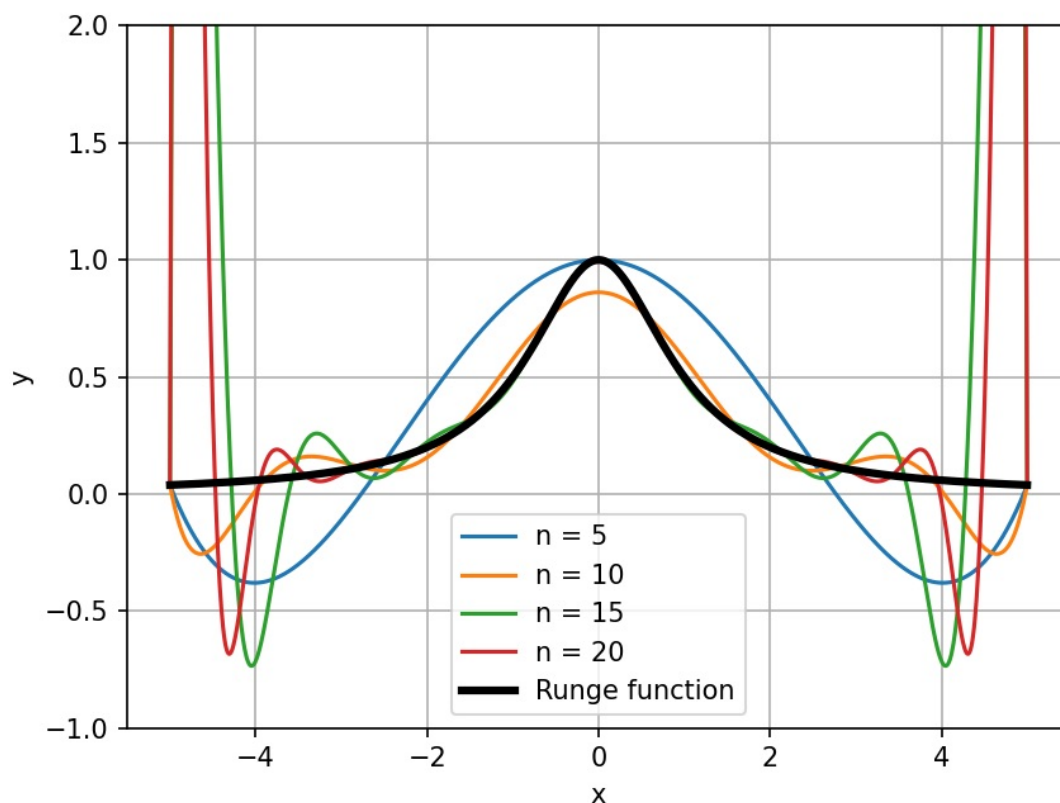
This only depends on the choice of interpolation nodes

What to keep in mind is small Λ_n is good!

Runge phenomenon:

When we interpolate a smooth function using one high-degree polynomial through equally spaced nodes, increasing the number of nodes doesn't really improve the approximation.

Code Example 3



So node selection is important!

↳ reason that we introduce Chebyshev Nodes/Interpolation!

Code expert: Clenshaw

We know high-degree interpolation at equidistant nodes can behave very badly
↳ so instead of changing the degree we change the distribution of the interpolation nodes

Chebyshev polynomials of the first kind

They are defined on $[-1, 1]$ by $T_n(x) = \cos(n \arccos x)$

⇒ How are these polynomials, we say $x = \cos \theta$

Then $T_n(x) = \cos(n\theta)$, for example:

$$T_1(x) = \cos(\arccos x) = x$$

$$T_2(x) = \cos(2\theta) = 2\cos^2\theta - 1 = 2x^2 - 1$$

Chebyshev polynomials of the second kind

For $x \in [-1, 1]$:
$$U_n(x) = \frac{\sin((n+1)\arccos x)}{\sin(\arccos x)}$$

→ Some properties:

- $T_0(x) = 1$; $T_1(x) = x$; $T_2(x) = 2xT_1(x) - T_0(x)$

⇒ Recurrence formula: $T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x)$

- As $x \in [-1, 1]$ ⇒ $|T_n(x)| \leq 1$

* Two useful sets of Chebyshev nodes

- The $n+1$ **Chebyshev nodes** on $[-1, 1]$ are the zeros/roots of $T_{n+1}(x)$

⇒ for arbitrary $[a, b]$:
$$x_k = a + \frac{1}{2}(b-a) \left(\cos\left(\frac{2k+1}{2(n+1)}\pi\right) + 1 \right) \quad k=0, \dots, n$$

- The **Chebyshev abscissae** are the extrema of T_n , which are zeros of U_{n-1} .

⇒ for arbitrary $[a, b]$:
$$x_k = a + \frac{1}{2}(b-a) \left(\cos\left(\frac{k}{n}\pi\right) + 1 \right) \quad k=0, \dots, n$$

Geometric intuition

→ To map a point $s \in [-1, 1]$ to $[a, b]$ use:

$$x = a + \frac{b-a}{2} (s+1)$$

• $(s+1) \in [0, 2]$ then $\frac{b-a}{2} (s+1) \in [0, b-a]$

at last $a + \frac{b-a}{2} (s+1) \in [a, b]$

→ We know $T_{n+1}(x) = \cos((n+1) \arccos x)$ we find its roots by $T_{n+1}(x) = 0$

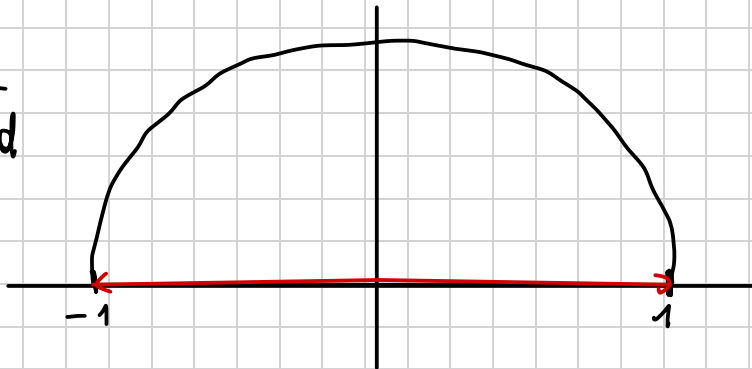
Let $\theta = \arccos x \Rightarrow \cos((n+1)\theta) = 0$

Cosine is zero at $\frac{\pi}{2}, \frac{3\pi}{2}, \frac{5\pi}{2} \Rightarrow (n+1)\theta_k = \frac{2k+1}{2} \pi$

$$\Rightarrow \theta_k = \frac{2k+1}{2(n+1)} \pi$$

$\Rightarrow x_k = \cos(\theta_k)$... roots of $T_{n+1}(x)$

→ Why are endpoints not in first-kind nodes?



Angle at endpoints are 0 and π respectively.

• But $T_{n+1}(1) = \cos((n+1) \cdot 0) = 1$
 $T_{n+1}(-1) = \cos((n+1) \cdot \pi) = \pm 1$ } $\neq 0 \Rightarrow$ so they are not included

$\Rightarrow$ Endpoints are not first-kind nodes

→ Code example 4

p in Chebyshev basis

$$p(x) = c_0 + c_1 T_1(x) + c_2 T_2(x) + \dots + c_n T_n(x)$$

→ chebexp finds c_k 's for us, do not change it.

Cleinschaw algorithm

We have $p(x) = c_0 + c_1 T_1(x) + c_2 T_2(x) + \dots + c_n T_n(x)$

→ set $d_{n+2} = d_{n+1} = 0$, then calculate backwards:

$$d_k = c_k + 2x d_{k+1} - d_{k+2} \quad k = n, n-1, \dots, 0$$

$$\Rightarrow p(x) = d_0 - x d_1$$

Where does it come from?

We use $T_{k+1} = 2x T_k - T_{k-1}$:

$$p(x) = c_0 + c_1 T_1(x) + \dots + c_{n-2} T_{n-2}(x) + c_{n-1} T_{n-1}(x) + \underbrace{c_n}_{d_n} T_n(x)$$

$2x T_{n-1} - T_{n-2}$

$$= c_0 + c_1 T_1(x) + \dots + (c_{n-2} - d_n) T_{n-2}(x) + \underbrace{(c_{n-1} + 2x d_n)}_{d_{n-1}} T_{n-1}(x)$$

$$= c_0 + c_1 T_1(x) + \dots + (c_{n-3} - d_{n-1}) T_{n-3}(x) + \underbrace{(c_{n-2} + 2x d_{n-1} - d_n)}_{d_{n-2}} T_{n-2}(x)$$

$$= d_0 - x d_1$$

⇒ recursion formula is clear here

Cooling tips

- Vectorization is needed as we are evaluating at more than one point
- Go backwards:

→ at each step compute $\underline{d_k} = c_k + 2x \underline{d_{k+1}} - d_{k+2}$

→ next step $k \rightarrow k-1$ ⇒ means you have to shift the variables

$$\Rightarrow d_{k-1} = c_{k-1} + 2x \underline{d_k} - \underline{d_{k+1}}$$

Discussion = Convergence

	Algebraic	Exponential
error	Cn^{-p}	Cq^n , $0 < q < 1$
straight line on	log-log plot	semilog plot
speed	slow	very fast

→ for very smooth functions

$$\begin{aligned} \bullet \log(Cn^{-p}) &= \log C - p \log n = \log(\epsilon_n) \\ \bullet \log(Cq^n) &= \log C + n \log q = \log(\epsilon_n) \end{aligned} \quad \left. \vphantom{\begin{aligned} \bullet \log(Cn^{-p}) &= \log C - p \log n = \log(\epsilon_n) \\ \bullet \log(Cq^n) &= \log C + n \log q = \log(\epsilon_n) \end{aligned}} \right\} \begin{array}{l} \text{and for both we plot} \\ n \text{ against the error} \end{array}$$

Code expert: 3-term Rekursion

→ They already gave the forward recursion as $y_n = 1 - n y_{n-1}$
with $y_0 = 1 - \frac{1}{e}$

→ For backwards look at the bound given as $0 \leq y_n \leq \frac{1}{N+1}$, they say take the starting value $y_n = 0$ and run backwards with $y_{n-1} = \frac{1 - y_n}{n}$

Output: No plot this time.

→ You must observe for forward recursion a disruption at around y_{18} .

→ For backwards recursion everything is stable.

Forward Recursion

• $\tilde{y}_{n-1} = y_{n-1} + e_{n-1}$ where e_{n-1} is small numerical error.

$$\begin{aligned} \rightarrow \tilde{y}_n &= 1 - n \tilde{y}_{n-1} \\ &= 1 - n (y_{n-1} + e_{n-1}) \\ &= 1 - n y_{n-1} + n e_{n-1} \end{aligned}$$

→ Therefore $|e_n| = n |e_{n-1}| \Rightarrow |e_n| \sim n! |e_0|$

Backward Recursion

Do the same as above for backward recursion.